RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ..... Naveen. S .....

ACADEMIC YEAR 2026-2027.. SEMESTER .....V.....BRANCH..CSD.

UNIVERSITY REGISTER No. 241701038

Certified that this is the bonafide record of work done by the above student in t

(IT23E31)

Graphic & Multimedia laboratory during the year 2026 - 2027

30/10/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examine

NAVEEN.S

III DIV: CSD ROLL NO.: 2417010038

CT: Graphics and multimedia

YOUVA

INDEX

DATE	TITLE	PAGE NO.	TEACHER'S SIGN
21.7.26	Bresenham's line drawing for |m|<1		
28.7.26	Bresenham's algorithm for circle		
28.7.26	Bresenham's algorithm for ellipse		
04.8.26	DDA line algorithm		
11.08.26	Bresenham Rectangle.		
11.08.26	Traffic signal.		
11.08.26	Smiling face.		
11.08.26	Draw dympic logo using ellypse.		
08.09.26	Scan line algorithm		
08.09.26	Flood fill algorithm		
08.09.26			
22.09.26	Basic 2D - Transformations		
	Polygon clipping		
	3D Transformations		
	create & Render 3D objects		
	Multimedia Application.		
	Capture Audio/Video from webcam.		
	Send & Recieve multimedia message.		
	3D Modelling using Blender.		

completed

Experiment - 1:

Bresenham line drawing for $|m| < 1$.

Aim:

To implement Bresenham line drawing for $|m| < 1$ using Python program

Code:

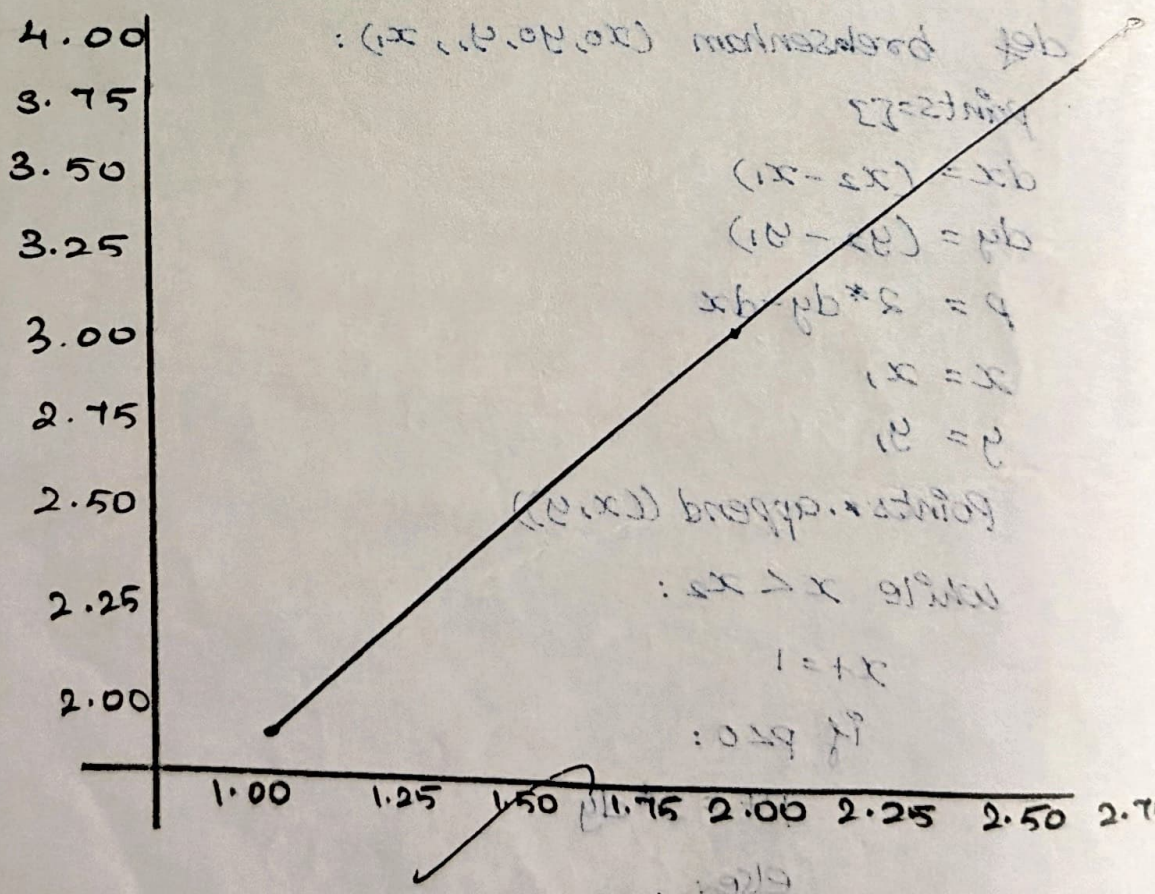
```
import matplotlib.pyplot as plt
def brehsenham (x0, y0, y1, x1):
 points=[]
 dx = (x2 - x1)
 dy = (y2 - y1)
 p = 2*dy-dx
 x = x1
 y = y1
 Points.append ((x, y))
 while x < x2:
     x+=1
     if p<0:
         p+ = 2*dy
         else:
             y+=1
             p+ = 2*dy - 2*dx
```

OUTPUT:

Enter x_1: 1
Enter y_2: 2
Enter x_2: 3
Enter y_2: 4

Points generated

(1, 2)
(2, 3)
(3, 4)



o/p verified

```
    points.append((x,y))
    return points
x1 = int(input("Enter x1:    "))
y1 = int(input("Enter y1:    "))
x2 = int(input("Enter x2:    "))
y2 = int(input("Enter y2:    "))
Points = bresenham_line(x1, y1, x2, y2)
print("points generated:")
for point in points:
    print(points)
x_points = [point[0] for point in points]
y_points = [point[1] for point in points]
plt.plot(x_points, y_points, marker='o')
plt.title("Bresenham Line Drawing
                Line")
plt.xlabel('x-axis')
plt.ylable('y-axis')
plt.grid()
plt.show()
```

21/7/26

Result:

The output generated Successfully.

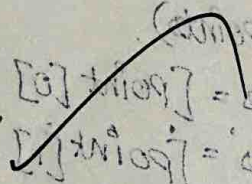
OUTPUT:

Enter Centre x-coordinate : 0

Enter Centre y-coordinate : 0

Enter radius: 5

Brenseham circle drawing.



Result:

The output generated successfully.

Experiment-2:

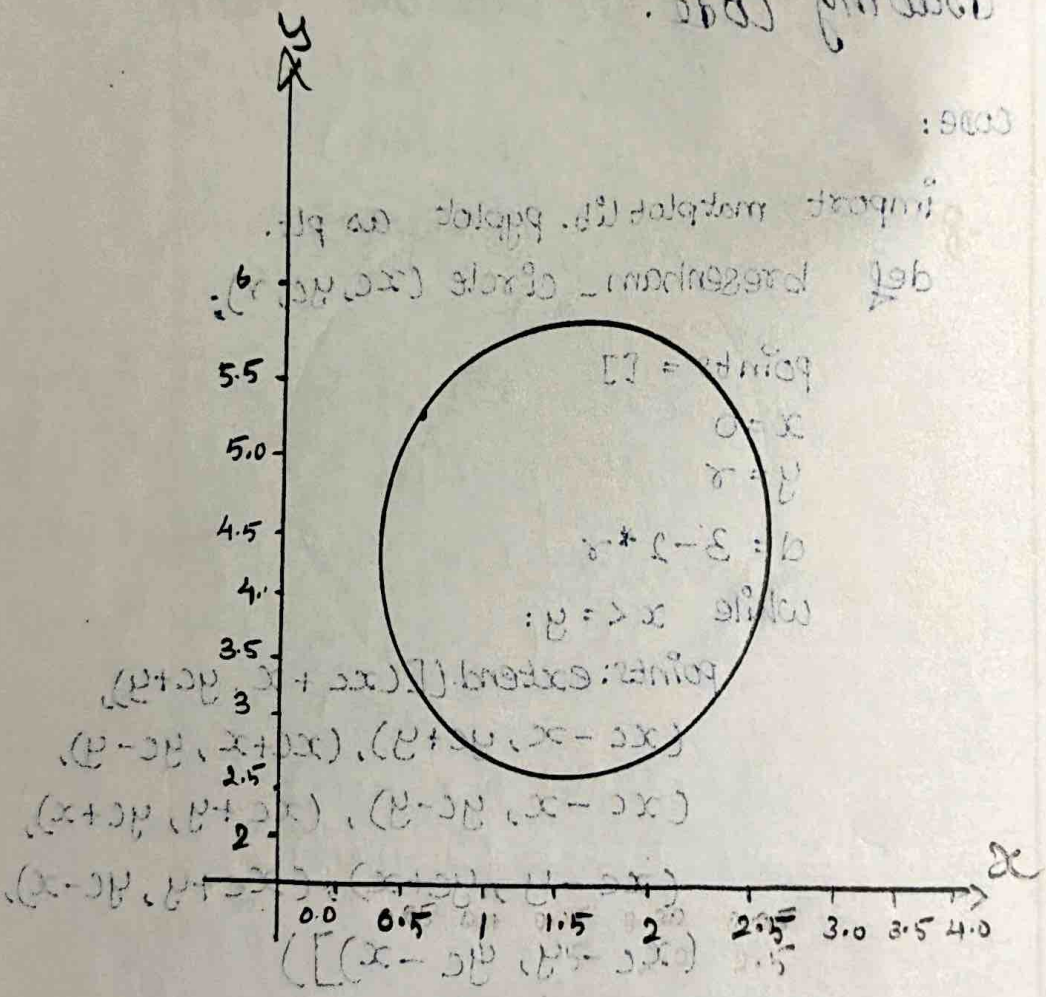
Bresenham CIRCLE CODE:

AIM:

To implement Brenshensam circle drawing code.

CODE:

```
import matplotlib.pyplot as plt.
def bresenham_circle (xc, yc, r):
    points = []
    x = 0
    y = r
    d = 3 - 2 * r
    while x <= y:
        points.extend ([[(xc + x, yc + y),
          (xc - x, yc + y), (xc + x, yc - y),
          (xc - x, yc - y), (xc + y, yc + x),
          (xc - y, yc + x), (xc + y, yc - x),
          (xc - y, yc - x)]])
        if d < 0:
            d = d + 4 * x + 6
        else:
            d = d + 4 * (x - y) + 10
            y -= 1
        x += 1
    return points
xc = int (input("enter centre x-coordinate: "))
yc = int (input("Enter centre y-coordinate: "))
r = int (input("enter radius: "))
```



Input :

$x_c : 2$

$y_c : 2$

$r = 4$

Points: (2,6) (2,6) (2,-2) (2,-2) (6,2)

(-2,2) (6,2) (-2,2) (6,2) (-2,2)

(3,6) (1,6) (3,-2) (1,2) (6,3) (4,5)

(-2,1)

```
points = bresenham_circle(xc, yc, r)
points = list(set(points))
print("Points generated: ")
for point in sorted(points):
    print(point)
```

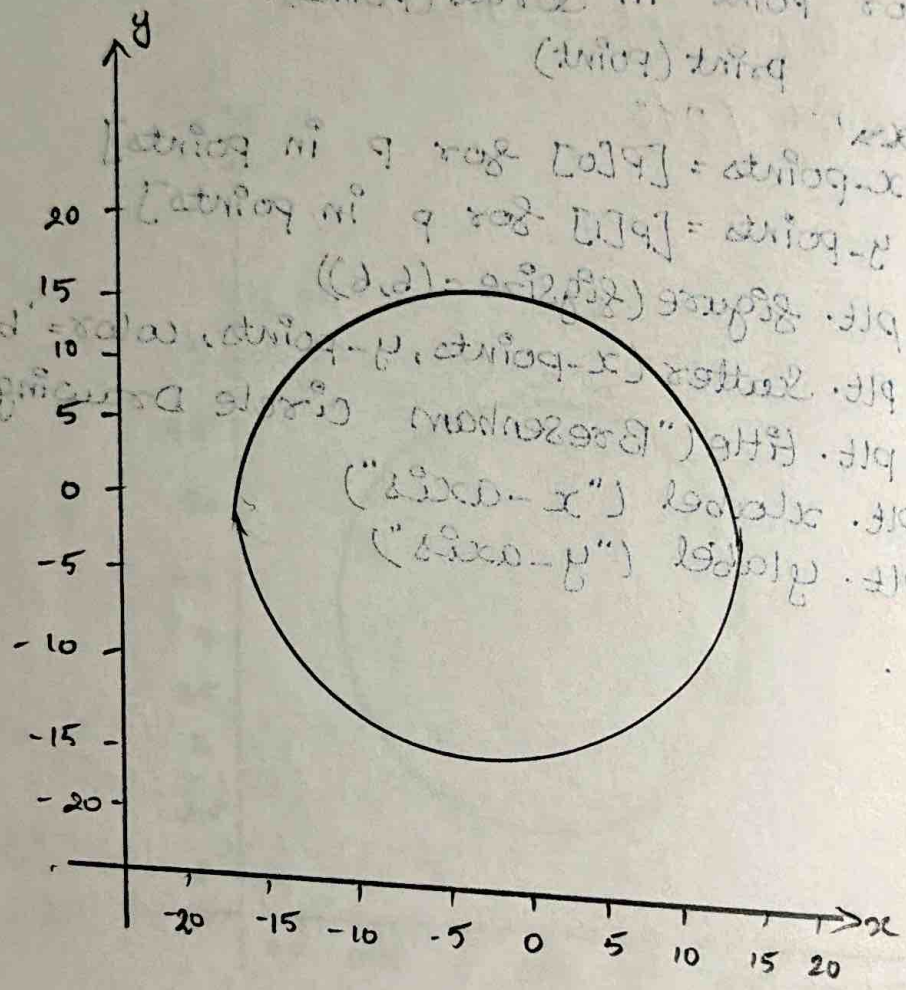
```
x_points = [p[0] for p in points]
y_points = [p[1] for p in points]
plt.figure(figsize=(6,6))
plt.scatter(x_points, y_points, color='blue')
plt.title("Bresenham circle Drawing")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
```

4/8/26

Result:

Thus to draw circle for Brehensam equation is implemented successfully

Output:



Enter centre x: 0

Enter Centre y: 0

Enter x radius: 20

Enter y radius: 10

Experiment-3.

Brehensam Ellipse code:

Aim:

To implement Brehensam ellipse drawing code.

code:

```
import matplotlib.pyplot as plt.
def plot_ellipse_points (xc, yc, x, y, points):
points.extend ([(xc+x, yc+y),
    (xc-x, yc+y), (xc+x, yc-y), (xc-x, yc-y)
def brenhensam_ellipse (xc, yc, rx, ry):
    points=[]
    x=0
    y=ry
    rx^2 = rx * rx
    ry^2 = ry * ry
    dx = 2 * ry^2 * x
    dy = 2 * rx^2 * y
    P1 = ry^2 - (rx^2 * ry) + (0.25 * rx^2)
    while dx<dy:
        plot-ellipse-points (xc, yc, x, y, points)
```

```
if p1<0:
    x+=1
    dx+=2*ry^2
    p1+=dx+ry^2
else:
    x+=1
    y-=1
    dx+=2*ry^2
    dy+=2*rx^2
    p1+=dx-dy+ry^2
p2=(ry^2*(x+0.5)**2)+
   (rx^2*(y-1)**2)-(rx^2*ry^2)
while y>=0:
    plot_ellipse_points(xc,yc,x,y,points)
    if p2>0:
        y-=1
        dy-=2*rx^2
        p2+=rx^2-dy
    else:
        y-=1
        x+=1
        dx+=2*ry^2
        dy-=2*rx^2
        p2+=dx-dy+rx^2
    return points
```

```
xc = int(input("Enter centre x: "))
yc = int(input("Enter centre y: "))
rx = int(input("Enter x radius: "))
ry = int(input("Enter y radius: "))
points = brenhensam_ellipse(xc, yc, rx, ry)
x = [p[0] for p in points]
y = [p[1] for p in points]
plt.figure(figsize=(6,6))
plt.scatter(x, y, s=20)
plt.title("Brenhensam Ellipse")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.axis("equal")
plt.show()
```

4/8/24

Result:

Thus to draw ellipse for Brehensam equation is implemented successfully.

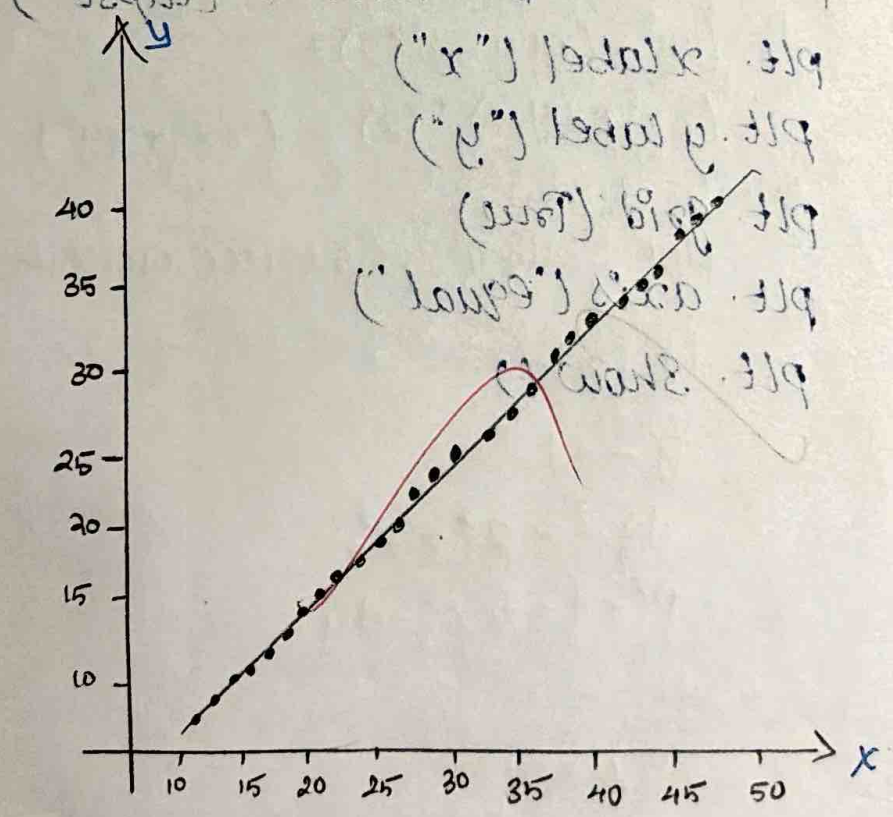
Output:

Enter X_1: 10

Enter y_1: 10

Enter X_2: 50

Enter y_2: 40



Result:

Thus to draw ellipse for Bresenham equation is implemented successfully.

Exp: 4

DDA algorithm for implementing object. Line drawing

Aim:

To implement the line drawing algorithm using DDA algorithm.

Code:

```
import matplotlib.pyplot as plt
def DDAline(x1, y1, x2, y2):
dx = x2 - x1
dy = y2 - y1
steps = max(abs(dx), abs(dy))
x_increment = dx/steps
y_increment = dy/steps
x = x1
y = y1
x_points = []
y_points = []
for i in range(steps+1):
    x_points.append(round(x))
    y_points.append(round(y))
    x+ = x_increment
    y+ = y_increment
```

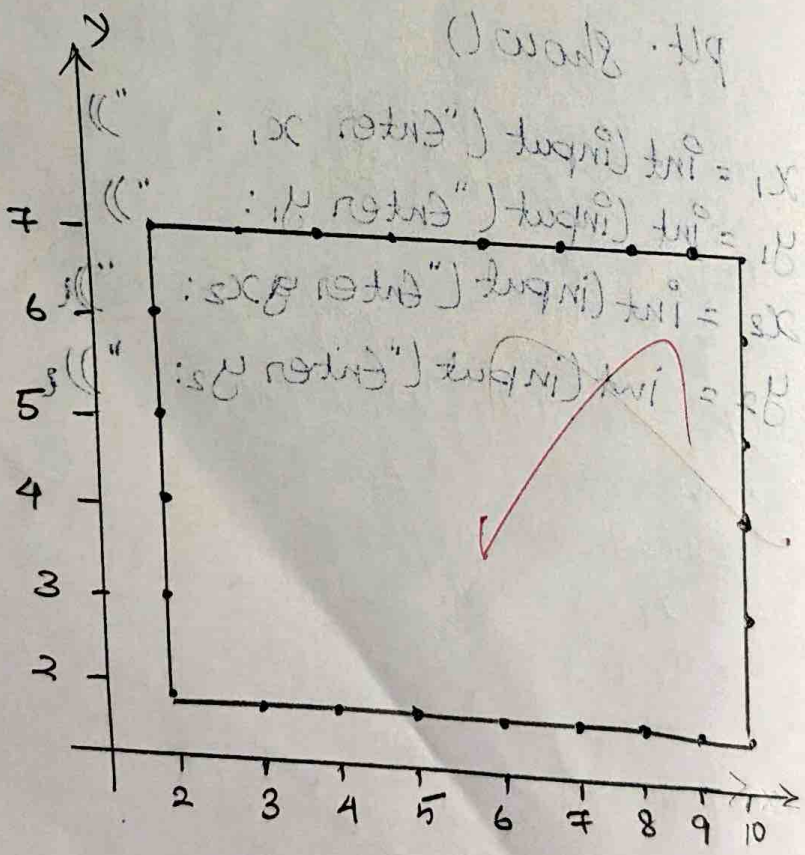
```
plt.figure(figsize=(6,6))
plt.plot(x_points, y_points, color='blue',
         linewidth=2, marker='o')
plt.title("DDA line drawing algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.gca().set_aspect("equal", adjustable
  ='box')
plt.show()
```

```
x1 = int(input("Enter x1:    "))
y1 = int(input("Enter y1:    "))
x2 = int(input("Enter x2:    "))
y2 = int(input("Enter y2:    "))
```

Result:

Thus to draw ~~line~~ ~~des~~ dda line ~~al~~ drawing algorithm is implemented successfully.

Output:



Result:

Exp: 5 : Rectangle using Bresenham algorithm.

Aim:

To Draw a Rectangle using Bresenham algorithm using Top to left (2,2) and width = 8, height = 5.

Code:

```
import matplotlib.pyplot as plt
def bresenham (x0, y0, x1, y1):
    points = []
    dx, dy = abs(x1 - x0), abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    while true:
        points.append ((x0, y0))
        if x0 == x1 and y0 == y1:
            break
        e = 2 * err
        if e > -dy
            err -= dy
            x0 += sy
        if e < dx:
            err += dx
            y0 += sy
    return points.
```

```
x,y = 2, 2
w,h = 8, 5

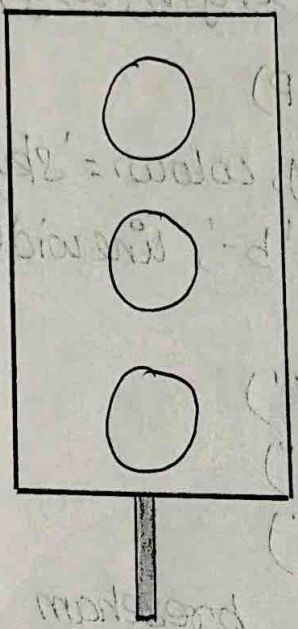
p = (bresenham(x, y, x+w, y) + bresenham
    (x+w, y, x+w, y+h) + bresenham (x+w, y+h, x,
                                    y+h),
    bresenham (x, y+h, x, y))

px, py = zip(*p)
plt.fill (px, py, colour = 'skyblue', alpha = 0.5)
plt.plot (px, py, 'b-', linewidth = 3)
plt.grid ()
plt.axis ('equal')
plt.xlabel ("x")
plt.ylabel ("y")
plt.title ("2D breseham Rectangle")
plt.show ()
```

Result:

Thus to draw Rectangle using Brehensam algorithm is implemented Successfully.

Output:



Result:

Thus to draw Rectangle using Bresenham algorithm is Implemented Successfully

Exp. #6: Brehensam Diagram for Traffic Signal.

Aim:

To draw Traffic signal using Bresenham Algorithm.

Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots(figsize=(5,8))
ax.add_patch(Rectangle((2,1),2,6, facecolor=
                color, edgecolor='black', linewidth=3,
            alpha=0.8))
ax.plot([3,3],[1,0],'k', linewidth=6)
ax.set_xlim(0,6)
ax.set_ylim(0,8)
ax.set_aspect('equal')
ax.axis('off')
plt.title("Traffic SPgnal")
plt.show()
```

Result:

The output generated successfully.

Exp. #1 : Representation of Traffic Signal.

Aim:

To show traffic signal using Python Libraries.

Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots(figsize=(5,8))
ax.add_patch(Rectangle((2,1),2,6, facecolor=
    color='gray', edgecolor='black', linewidth=3,
    zorder=0.8))
ax.plot([3,3],[1,0.2], 'k', linewidth=5)
ax.set_xlim(0,5)
ax.set_ylim(0,8)
ax.set_aspect('equal')
ax.axis('off')
plt.title("Traffic Signal")
plt.show()
```


Result:

The output generated successfully.

Exp:7. Smiling face using Brehensan algorithm.

aim:

To draw Smiling face using Bresenham algorithm.

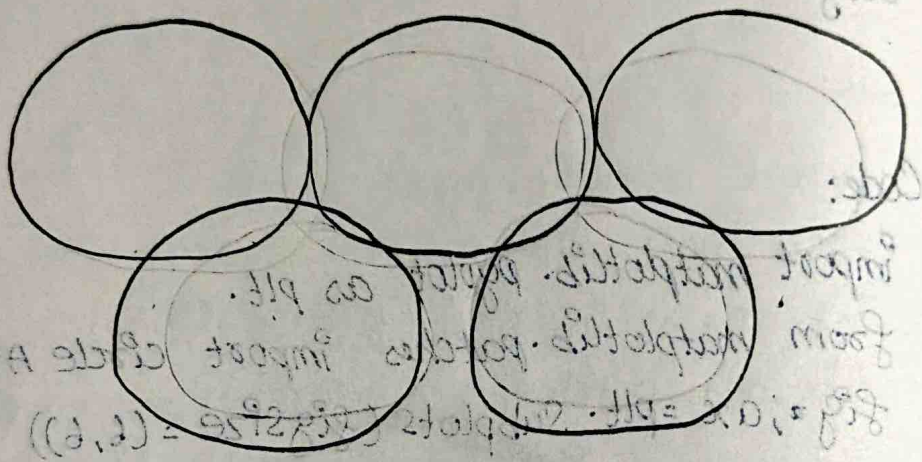
Code:

```
import matplotlib.pyplot as plt.
from matplotlib.patches import Circle Arc
fig, ax = plt.subplots(figsize = (6,6))
ax.add_patch(Circle((5,5),3, facecolor = 'gold',
        edgecolor = 'black', linewidth = 3))
ax.add_patch(Circle((4,6),0.3, color = 'black'))
ax.add_patch(Circle((6,6),0.3, color = 'black'))
ax.add_patch(Circle(5,4.8),0.12, color = 'black'))
ax.add_patch(Arc(5,4.3),2,2.2, theta1=180, theta2=360,
        color='black', linewidth = 3))
ax.set_xlim(1,9)
ax.set_ylim(1,9)
ax.set_aspect("equal")
ax.axis('off')
plt.title("Smiling face")
plt.show().
```

Result:

The output generated successfully.

Output:



Exp.8. Olympic Logo

Aim:

To Draw olympic logo using ellypse

code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
fig, ax = plt.subplots (figsize = (8,5))
rings = [(2,3,'blue'), (5,3,'black'), (8,3,'red'),
(3.5,1.5,'yellow'), (6.5,1.5,'green')
for x,y,c in rings:
    ax.add_patch (Ellipse ((x,y),3,2, fill=False,
                  edgecolor=c, linewidth=5))
ax.set_xlim (0,10)
ax.set_ylim (0,5)
ax.set_aspect('equal')
ax.axis ('off')
plt.title (" olympic Logo")
plt.show.
```

Result:

The output generated Successfully.

Exp: 9 Scan line algorithm:

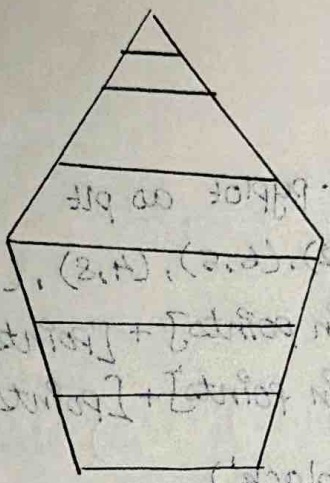
Aim:

To draw Scan line algorithm using pentagon.

Code:

```
import matplotlib.pyplot as plt
points = [(2,2),(5,2),(6,5),(4,8),(1,5)]
x= [p[0] for p in points] + [points[0][0]]
y=[p[1] for p in points]+[points[0][1]]
plt.plot(x,y,'black')
y min =min(y)
y max = max(y)
for scan-y in range(y min, ymax+1):
 intersection =[]
    for i in range(len(points)):
        x1, y1 = points[i]
        x2, y2 = points[(i+1) % len(points)]
        if(y1 <= scan-y < y2) or (y2 <=scan-y< y1):
        x_intersect = x1+(scan-y-y1)*(x2-x1)/(y2-y1)
        intersections.append(x_intersect)
    intersections.sort()
    for i in range(0,len(intersections),2):
      if i+1 < len(intersections):
       plt.plot([
          [intersections[i], intersection[i+1]],
          [scan-y, scan-y]'b')
```

Output:



```
plt.xlim(0,8)
plt.ylim(0,8)
plt.gca().set_aspect('equal')
plt.title("Pentagon using Scan line
          algorithm")
plt.grid()
plt.show()
```

Result:

To draw Scan line algorithm using Pentagon is implemented Successfully.

Ex: 10. Flood fill algorithm.

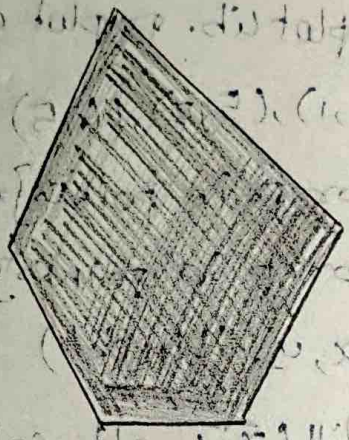
Aim:

To draw pentagon using flood fill algorithm.

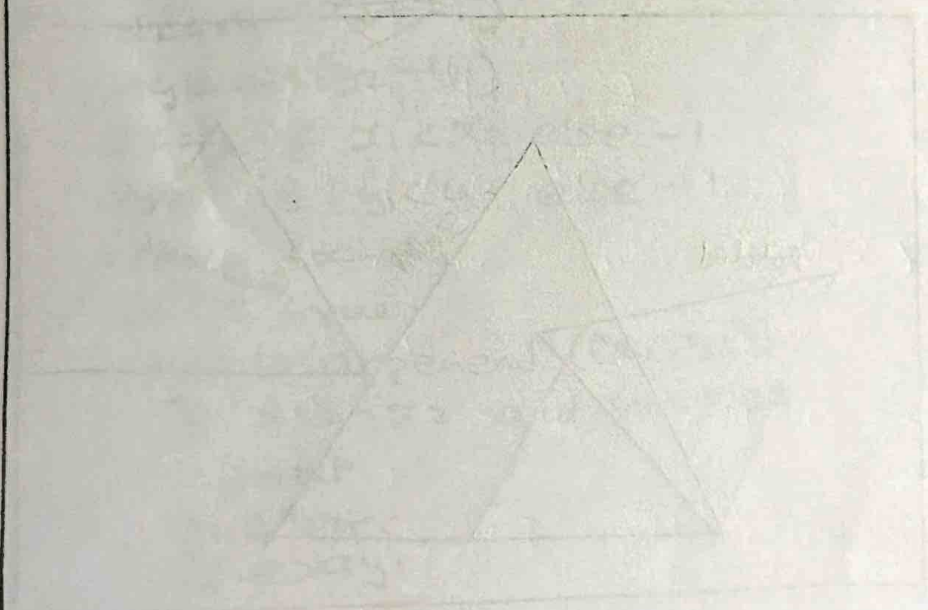
Code:

```
import matplotlib.pyplot as plt.
Points = [(2,1),(5,2),(6,5),(4,7),(1,5)]
x=[p[0] for p in points]+[Points[0][0]]
y=[p[1] for p in points]+[Points[0][1]]
plt.plot(x,y,'black')
def flood-fill(x,y,old-color,new-color):
    if (old-color == new-color):
        return
    if x<0 or x>=100 or y<0 or y>=100:
        return
    if pixels[x][y] != old-color:
        return
    pixel[x][y] = new-color.
    flood-fill(x+1, y, old-color, new-color)
    flood-fill(x-1, y, old-color, new-color)
    flood-fill(x, y+1, old-color, new-color)
    flood-fill(x, y-1, old-color, new-color)
pixel=[[0 for j in range(100)] for i in
                                range(100)]
plt.fill(x,y,'skyblue")
plt.xlim(0,8)
plt.ylim(0,8)
```

Output:



```
plt.gca().set_aspect('equal')
plt.title("Pentagon using flood fill algorithm")
plt.show().
```



Result:

Thus to draw pentagon using flood-fill algorithm is generated successfully.

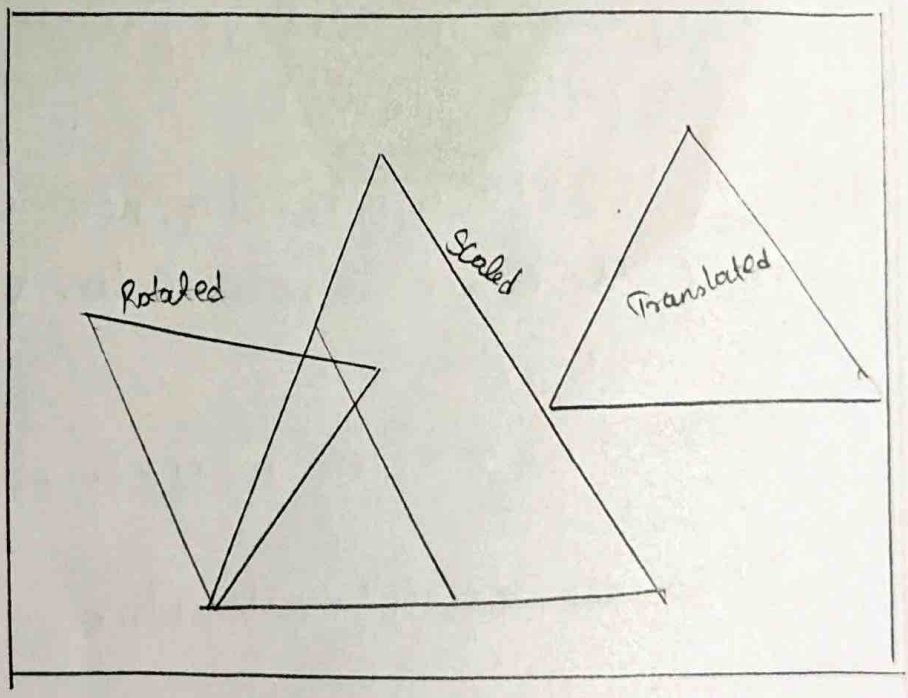
Output:

1. Transformation
2. Rotation
3. Scaling.

Enter choice : 1

Enter tx : 20

Enter ty : 60



Ex:11. 2D Transformations:

Aim:

To write a program that performs.

1. Scaling.
2. Rotating
3. Rotation.

Code:

```
import matplotlib.pyplot as plt.
import math
def bresenham (x1, y1, x2, y2)
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx - dy
    while true:
      points.appenend ((x1, y1))
      if x1 == x2 and y1 == y2:
        break
      e = 2*err
      if e > ay:
        err -= ay
        x1 += sx
      if e < ax:
        err += ax
        y1 += sy
      return points
    def draw_polygon (polygon_label):
    for i in range(1).
```

Result:

Thus, the output generated Successfully.

Ex: 12: Polygon clipping using Sutherland-Hodgman Algorithm.

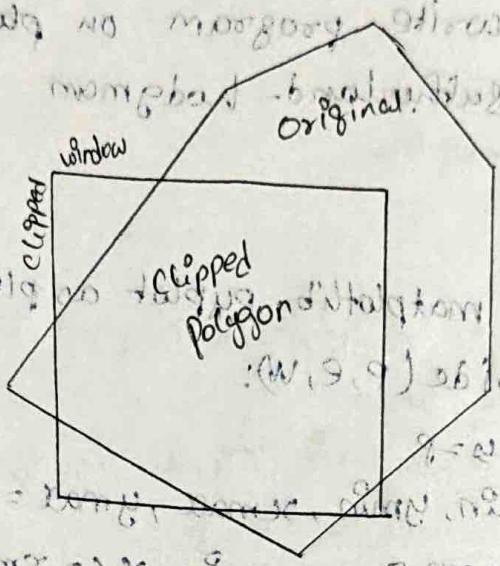
Aim:

To write program on polygon-clipping using Sutherland-Hodgman Algorithm.

Code:

```
import matplotlib.pyplot as plt.
def inside(p,e,w):
    x,y=p
    xmin, ymin, xmax, ymax = w
    return [x>=xmin, x<=xmax,
            y>=ymin, y<=ymax][e]
def intersect(a,b,e,w):
    xmin, xmax, ymin, ymax=w
    x1,y1=a
    x2,y2=b
    if e<2:
        x=[xmin, xmax][e]
        y= y1+(y2-y1)*(x-x1)/(x2-x1)
    else:
        y=[ymin, ymax][e-2]
        x= x1+(x2-x1)*(y-y1)/(y2-y1)
    return x,y
def clip(poly, w)
    for e in range(4):
        inp,out=poly,[]
        if not inp:
            break
        s=inp[-1]
```

Output:



```
    for p in inp:
        if inside (p,e,w):
            if not inside (s,e,w):
                out.append (intersect (s,p,e,w))
            out.append [p]
        elif inside (s,e,w):
            out.append (intersect (s,p,e,w))
            s=p
        poly= out
        return poly.
def draw(p, color, label):
    if p:
        x,y = zip(*p+[p[0]])
        plt.plot(x,y,color= color, label=label)
w= (100, 300, 100, 300)
poly=[(50,150),(200,50),(350,150),
      (350,300),(250,350),(150,300)]
draw (poly, 'blue', 'original')
draw ([(100,100),(300,100),(300,300),(100,300)],
  'black', 'window')
draw (clip (poly, w), 'red', 'clipped')
plt.legend()
plt.grid()
plt.axis('equal')
plt.show().
```

Result:

The output generated successfully.

Exp: 13: 3D Transformations on Basic objects.

Aim:

To write a program on 3D ~~basic~~ Transformation on Basic objects.

Code:

```
import numpy as np.
import matplotlib.pyplot as plt.
from mpl_toolkits.mplot3D.art3D
 import 3Dcollection.
def draw_edges(ax,v,e,c):
  ax.add_collection3D(Line3Dcollection(
                [[v[i],v[j] for i,j in e],
                 colors=c))
  def transform(v,M)
    return[(M@np.array([*p,1]))[:3] for p in v]
vertices=[[0,0,0),(1,0,0),(1,1,0),(0,1,0),
          (0,0,1),(1,0,1),(1,1,1),(0,1,1)]
edges = [(0,1),(1,2),(2,3),(3,0),(4,5),(5,6)
         (6,7),(7,4),(0,4),(1,5),(2,6),(3,7)]
t=np.array([[1,0,0,2],[0,1,0,2],[0,0,1,0],
            [0,0,0,1]])
s=np.diag([1.5,1.5,1.5,1])
a= np.radians(45)
r= np.array([[np.cos(a),-np.sin(a),0,0],
             [np.sin(a),np.cos(a),0,0],
             [0,0,1,0],[0,0,0,1]])
new=transform(vertices, t@s@r)
fig=plt.figure()
```

Exp: 12 3D Transformations on basic objects.

Aim:

To write a program on 3D transformation on basic objects.

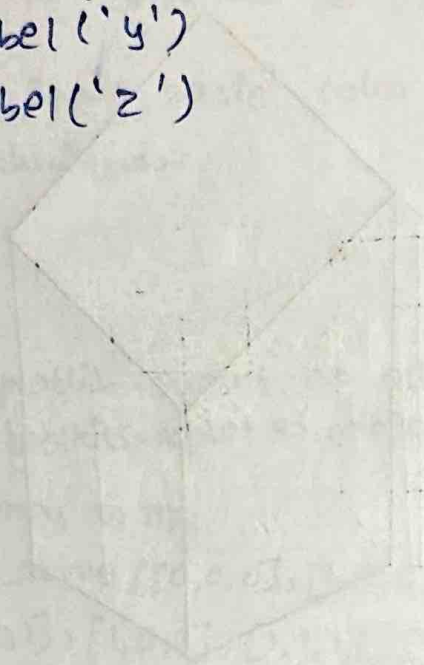
output:

code:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import 3D collection.
def draw_cube(ax, c):
    ax.add_collection3d(Poly3DCollection([V[i] for i in f],
                                         facecolors=c))
def transform(V, M):
    return [(M @ np.append(v, 1))[:3] for v in V]
vertices = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0),
            (0, 0, 1), (1, 0, 1), (1, 1, 1), (0, 1, 1)]
edges = [(0, 1), (1, 2), (2, 3), (3, 0), (4, 5), (5, 6), (6, 7),
         (7, 4), (0, 4), (1, 5), (2, 6), (3, 7)]
t = np.pi/4
```

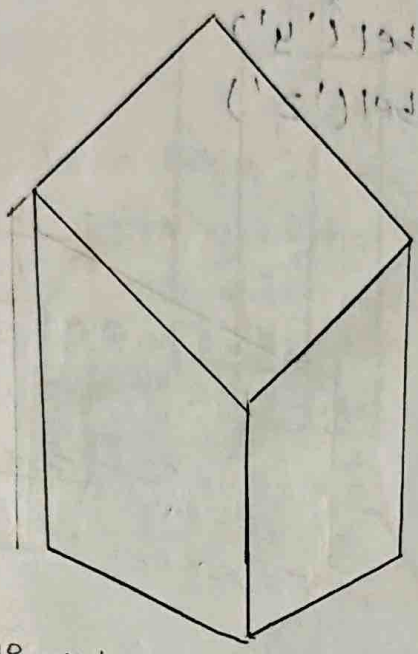
```
[0, 0, 0])
Scaling((1.5, 1.5, 1.5),
def = np.radians(45)
g = np.array([[np.cos(a), -np.sin(a), 0, 0],
[np.sin(a), np.cos(a), 0, 0],
[0, 0, 1, 0], [0, 0, 0, 1]])
def draw_cube(ax, c):
ax = plt.subplot(111, projection='3d')
```

```
ax = fig.add_subplot(111, projection='3d')
draw_edges(ax, vertices, edges, 'blue')
draw_edges(ax, new, edges, 'red')
ax.set_title('3D transformation of cube')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
plt.show().
```



Result:

The output generated successfully.



Coding Continues:

```
colors = ["red", blue, green, yellow, cyan, orange]
fig = plt.figure()
ax = fig.add_subplot(111, projection='3D')
poly3D = poly3Dcollection(faces, facecolors=colors,
                edgecolors='black', linewidth=1)
ax.add_collection3d(poly3d)
#Set view
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('z')
    ax.set_titles("3D cube with flat shading")
    ax.set_box_aspect([1,1,1])
    plt.show()
```

Exp: 14. Create and Render 3D objects using polygons and apply Basic color and Shading Techniques.

AIM:

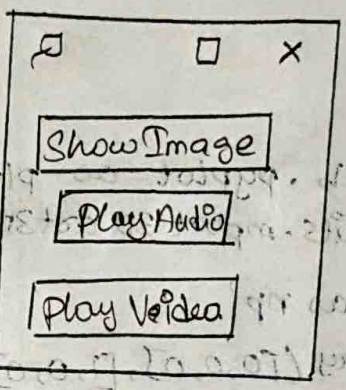
To create and render 3D objects using polygons and apply basic color and Shading techniques.

Program:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import
                                    Poly3DCollection
import numpy as np
vertices = np.array([0,0,0],[1,0,0], [1,1,0],[0,1,0],
        [0,0,1], [1,0,1], [1,1,1], [0,1,1])
faces = [[vertices[j] for j in [0,1,2,3]],
  [vertices[j] for j in [4,5,6,7]],
  [vertices[j] for j in [0,1,5,4]],
  [vertices[j] for j in [2,3,7,6]],
  [vertices[j] for j in [1,2,6,5]],
  [vertices[j] for j in [4,7,3,0]].
```

Result:

The output generated Successfully for create and render 3D objects using polygons.


Show Image
Play Audio
Play Video

Ex:15: Create a multimedia Application that Intergrates Images, Sound and ~~Vedio~~ Video In a Simple user Interface

Aim:

To develop a simple multimedia application using python.

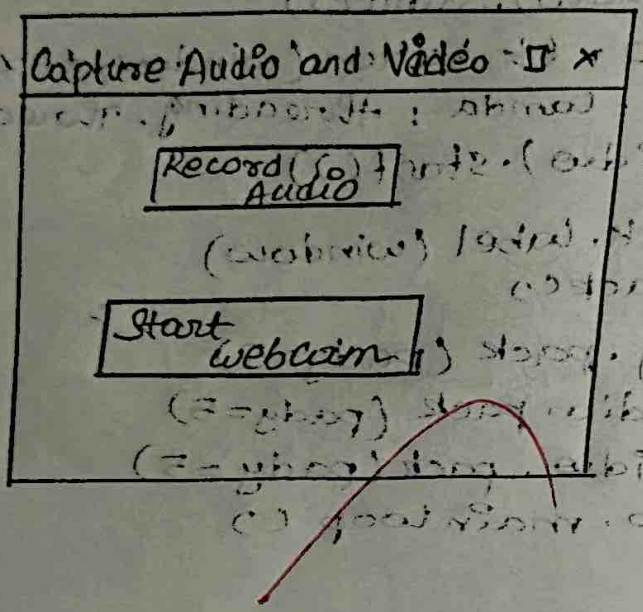
Code:

```
import tkinter as tk
from tkinter import fieldalong
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def play_audio():
    playsound('sample_audio.mp3')
def play_video():
    cap = cv2.VideoCapture('sample_video.mp4')
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imShow("Video", frame)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllwindows()
def load_image
    img = Image.open("Sample_image.jpg")
    img = img.resize((300, 300))
    img_tk = ImageTk.PhotoImage(img)
    Panel.configure(image = Image_tk)
    Panel.imgage = imag_tk.
```

```
window = tk.Tk()
window.title("Multimedia app")
btn_img = tk.button(window, text = 'show Image',
    command = load_image)
btn_audio = tk.button(window, text = 'play Audio',
    command = lamda: threading.Thread(target =
    play_audio).start())
btn_video = tk.button(window, text = "play Video",
    Command = lamda : threading.Thread(target =
    play_video).start())
Panel = tk.label(window)
Panel.pack()
btn_img.pack(pady=5)
btn_audio.pack(pady=5)
btn_vidio.pack(pady=5)
window.mainloop()
```

Result:

Thus the output has been verified for a simple multimedia application for Python.


Capture Audio and Video
Record Audio
Start Webcam

Ex. no: 16 Capture Video/Audio from webcam or microphone and Display on multimedia Interface.

Aim:

To develop a program that captures live video and live audio.

Code:

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk
def record_audio():
duration = 5
fs = 44100
print("Recording Audio.")
audio, sd.rec(int(duration * fs), samplerate = fs,
sd.wait()                          channel = 2)
write('recorded_audio.wav', fs, audio)
print("Audio recording saved.")
def capture_video():
    cap = cv2.VideoCapture(0)
    print("Press 'q' to stop video.")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow('Webcam Feed', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()

    window = tk.Tk()
    window.title("Capture Audio and Video")
btn_audio = tk.Button(window, text = 'Record Audio',
Command = lambda : Threading.Thread(target=
    record_audio).start())
```

```
btn_video = tk.Button(window, text="Start Webcam",
command=lambda: threading.Thread(target=capture_
video).start())
btn_audio.pack(pady=10)
btn_video.pack(pady=10)
window.mainloop()
```

Result:

Thus, the output has been verified for Program that captures live video and live audio.

Ex: 14.

Create an Application to send and Recieve Multimedia Messages (text, Image, Audio)

Aim:

To develop a basic chat-like application that allows text, images, audio.

Code:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading
def send_text():
    msg = text_entry.get()
    if msg:
        chat_box.insert(tk.END, f"You: {msg}\n")
        text_entry.delete(0, tk.END)
def send_image():
    path = filedialog.askopenfilename
    (filetypes=[("image files", "*.jpg *.png *.
                                    jpeg")])
    if path:
        chat_box.insert(tk.END, f"You sent an
                image: {path}\n")
        img = Image.open(path)
        img.thumbnail((100, 100))
        img_tk = ImageTk.PhotoImage(img)
        image_label = tk.Label(chat_frame,
                image = img_tk)
        image_label.image = img_tk
        image_label.pack()
        chat_box.yview(tk.END)
```

```
def send_audio():
    path = filedialog.askopenfilename(filetypes=
    [("Audio files", "*.mp3 *.wav")])
    if path:
        chat_box.insert(tk.END, f"You sent an
            audio message: {path}\n")
        threading.Thread(target=lambda:
        playsound(path)).start()
window = tk.Tk()
window.title("Multimedia Messaging App")
window.geometry("400x500")
chat_frame = tk.Frame(window)
chat_box = scrolledtext.ScrolledText(chat_frame,
    wrap=tk.WORD, width=50, height=20)
chat_box.pack()
chat_frame.pack(pady=10)
text_entry = tk.Entry(window, width=30)
text_entry.pack(side=tk.LEFT, padx=5)
btn_text = tk.Button(window, text="Send text",
    command=send_text)
btn_image = tk.Button(window, text="Send Image",
    command=send_Image)
btn_audio = tk.Button(window, text="Send audio",
    command=send_Audio)
btn_text.pack(side=tk.LEFT)
btn_image.pack(side=tk.LEFT)
btn_audio.pack(side=tk.LEFT)

window.mainloop()
```

Result:

Thus, the output has been verified for develop a basic chat application that allows text, images and audio.

Ex: 18

Create a 3D model using Blender and Apply Basic Shading and Textures.

Aim:

To create a simple 3D model using Blender

Procedure:

1. Launch Blender.
2. Model Creation:
   → use Add > mesh to insert basic shapes like cube, cylinder, plane, etc.
   → combine these objects to form a house or a similar structure.
   → use edit mode for extruding and shaping.
3. Shading:
   → Switch to Shading workspace.
   → Add a new material for each object.
   → Adjust Base color, Specular and Roughness.
4. Texture Mapping:
   → In the shader editor, add an image texture node.
   → Load an external image.
   → use UV editing to align the texture.
5. Lighting and camera:
   → Add light sources.
   → position the camera to frame the model.
6. Render settings:
   → choose cycles or eevee rendring Engine.
   → Adjust Resolution and sampling
   → click Render > Render Image to produce the output.

Result:

Thus to create a 3D simple model using Blender is successfully Implemented.